Mini Project on Office Employees Data

Source: https://github.com/PacktPublishing/50-Hours-of-Big-Data-PySpark-AWS-Scala-and-Scraping/tree/main/Part%203/Code/03-Spark%20DFs

Dataset: OfficeDataProject.csv

In [1]:
```
sc
```

Out[1]: **SparkContext**

Spark UI

Version	`v4.0.0`
Master	`local[*]`
AppName	`PySparkShell`

Dataset Insights

- **Total Rows:** 1000
- **Total Columns:** 7
- **Missing Values:** 0
- **Duplicate Rows:** 0

Column Descriptions

Column Name	Description	Example
employee_id	Unique identifier for each employee	1000
employee_name	Full name of the employee	Nitz Leif
department	Department where the employee works	Marketing
state	State code where the employee is located	CA
salary	Monthly salary of the employee (in USD)	6131
age	Age of the employee (in years)	26
bonus	Annual bonus amount (in USD)	543

Numerical Columns Summary

	count	mean	std	min	25%	50%	75%	max
employee_id	1000	1499.5	288.819	1000	1249.75	1499.5	1749.25	1999
salary	1000	5329.94	2603.21	1006	3095.25	5188	7614.75	9985
age	1000	35.316	8.88533	20	27.75	35	43	50
bonus	1000	1252.91	432.96	500	872	1244.5	1640.25	2000

Categorical Columns Unique Counts

Column	Unique Values
employee_name	1000
department	10
state	50

In [2]:
```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, min, max

# Start Spark session
spark = SparkSession.builder.appName("OfficeDataProject").getOrCreate()

# Read the CSV file
df = spark.read.option("header", True).option("inferSchema", True).csv("OfficeDataProject.csv")
```

1. Total number of employees

In [3]:
```
print("Total number of employees:", df.count())
```

```
Total number of employees: 1000
```

2. Total number of deaprtments

In [4]:
```
print("Total number of departments:", df.select("department").distinct().count())
```

```
Total number of departments: 6
```

3. Department Names

In [5]:
```
print("Department names:")
df.select("department").distinct().show()
```

```
Department names:
+----------+
|department|
+----------+
|     Sales|
|        HR|
|   Finance|
|Purchasing|
| Marketing|
|  Accounts|
+----------+
```

4. Total number of employees in each deaprtment

In [6]:
```
print("Employees in each department:")
df.groupBy("department").count().show()
```

```
Employees in each department:
+----------+-----+
|department|count|
+----------+-----+
|     Sales|  169|
|        HR|  171|
|   Finance|  162|
|Purchasing|  166|
| Marketing|  170|
|  Accounts|  162|
+----------+-----+
```

5. Total number of employees in each state

In [7]:
```
print("Employees in each state:")
df.groupBy("state").count().show()
```

```
Employees in each state:
+-----+-----+
|state|count|
+-----+-----+
|   LA|  205|
|   CA|  205|
|   WA|  208|
|   NY|  173|
|   AK|  209|
+-----+-----+
```

6. Total number of employees in each state in each department

In [8]:
```
print("Employees in each state in each department:")
df.groupBy("state", "department").count().show()
```

```
Employees in each state in each department:
+-----+----------+-----+
|state|department|count|
+-----+----------+-----+
|   CA|     Sales|   42|
|   CA| Marketing|   33|
|   NY|  Accounts|   34|
|   NY|     Sales|   27|
|   CA|   Finance|   35|
|   CA|  Accounts|   35|
|   CA|Purchasing|   32|
|   WA|        HR|   47|
|   AK|Purchasing|   30|
|   WA|  Accounts|   27|
|   WA|Purchasing|   38|
|   AK|     Sales|   38|
|   AK|  Accounts|   37|
|   WA| Marketing|   39|
|   LA|        HR|   41|
|   LA|     Sales|   35|
|   AK|        HR|   25|
|   LA|   Finance|   29|
|   AK|   Finance|   37|
|   LA|Purchasing|   45|
+-----+----------+-----+
only showing top 20 rows
```

7. Min and Max salaries n each department and sorted salaries

In [9]:
```
print("Minimum and maximum salaries in each department:")
df.groupBy("department").agg(min("salary").alias("min_salary"),
                             max("salary").alias("max_salary")).show()

print("Salaries sorted in ascending order:")
df.select("employee_name", "department", "salary").orderBy("salary").show()
```

Minimum and maximum salaries in each department:

department	min_salary	max_salary
Sales	1103	9982
HR	1013	9982
Finance	1006	9899
Purchasing	1105	9985
Marketing	1031	9974
Accounts	1007	9890

Salaries sorted in ascending order:

employee_name	department	salary
Juliana Kreamer	Finance	1006
Tamra Deandre	Accounts	1007
Dionne Kall	HR	1013
Ruzicka Phylicia	Accounts	1029
Lisabeth Luisa	Marketing	1031
Herder Baros	Finance	1053
Gallman Nakano	Accounts	1060
Marvis Lisabeth	Finance	1062
Figueras Yessenia	Marketing	1093
Hanh Tyree	HR	1093
Kensinger Antonina	Finance	1093
Lonergan Megan	Sales	1103
Kohn Figueras	Purchasing	1105
Shandra Trena	Purchasing	1116
Laub Antonina	Sales	1117
Yessenia Byrge	HR	1118
Dynes Kall	Purchasing	1121
Vivan Sifford	Finance	1129
Luisa Suzanne	Accounts	1151
Fender Coogan	Marketing	1151

only showing top 20 rows

8. Employees in NY under Finance dept with bonus > avg NY bonus

In [10]:
```
ny_avg_bonus = df.filter(col("state") == "NY").agg(avg("bonus").alias("avg_bonus")).collect()[0]["avg_bonus"]

print("Employees in NY Finance with bonus > average NY bonus:")
df.filter((col("state") == "NY") &
          (col("department") == "Finance") &
          (col("bonus") > ny_avg_bonus)) \
  .select("employee_name", "department", "bonus").show()
```

```
Employees in NY Finance with bonus > average NY bonus:
+--------------------+----------+-----+
|       employee_name|department|bonus|
+--------------------+----------+-----+
|       Vivan Sifford|   Finance| 1261|
|      Herder Gallman|   Finance| 1402|
|          Nena Rocha|   Finance| 1647|
|       Leif Lemaster|   Finance| 1782|
|Ellingsworth Meli...|   Finance| 1358|
|        Escoto Gilma|   Finance| 1285|
|     Georgeanna Laub|   Finance| 1679|
|     Durio Tenenbaum|   Finance| 1684|
|       Juliana Grigg|   Finance| 1617|
|        Tiffani Benz|   Finance| 1969|
|          Nitz Ilana|   Finance| 1342|
|   Phylicia Antonina|   Finance| 1857|
|         Durio Janey|   Finance| 1733|
|       Melissia Jere|   Finance| 1533|
|      Yukiko Kreamer|   Finance| 1332|
|      Nena Kensinger|   Finance| 1610|
|      Antonina Ilana|   Finance| 1718|
+--------------------+----------+-----+
```

9. Raise salaries by $500 for employees older than 45

In [11]:
```
df_updated = df.withColumn("salary", col("salary") +
                           (500 * (col("age") > 45).cast("int")))
```

```
print("Updated salaries for employees older than 45:")
df_updated.filter(col("age") > 45).select("employee_name", "age", "salary").show()
```

```
Updated salaries for employees older than 45:
+------------------+---+------+
|     employee_name|age|salary|
+------------------+---+------+
|       Tamra Amber| 47|  6217|
| Recalde Kensinger| 48|  4204|
|  Barringer Escoto| 49|  2185|
|Vankirk Jacquelyne| 47|  9136|
|   Dionne Lemaster| 48|  5634|
|        Trena Benz| 49|  4876|
|      Dynes Katlyn| 48|  3539|
|      Clune Norene| 49|  2105|
|      Rocha Dionne| 49|  3970|
|       Imai Locust| 49| 10482|
| Clemencia Rudolph| 50|  1796|
|    Zollner Marvis| 50|  4730|
|     Kohn Antonina| 48|  9811|
|     Norene Mayeda| 50|  3600|
|      Jaclyn Baros| 48|  2199|
|    Gallman Nakano| 47|  7313|
|       Luisa Grigg| 48|  9797|
|   Kreamer Shandra| 46|  5468|
|     Fender Coogan| 50|  1651|
|  Lonergan Bergren| 46|  8675|
+------------------+---+------+
only showing top 20 rows
```

10. Create DF of employees older than 45 and save to file

In [12]:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

# Create Spark session with nativeio disabled
spark = SparkSession.builder \
    .appName("OfficeDataProject") \
    .config("spark.hadoop.io.nativeio.enabled", "false") \
```

```
    .getOrCreate()

output_path = "file:///C:/Users/subha/PythonPractice/Big Data Analytics/Mini Project/Employees_Age_Above_45"

older_45_df = df_updated.filter(col("age") > 45)

# Save as a single CSV with header
try:
    older_45_df.coalesce(1) \
        .write.option("header", True) \
        .mode("overwrite") \
        .csv(output_path)
    print(f"Saved employees older than 45 into: {output_path}")
except Exception as e:
    print("Spark CSV write failed, using Pandas fallback...")
    older_45_df.toPandas().to_csv(
        "C:/Users/subha/PythonPractice/Big Data Analytics/Mini Project/Employees_Age_Above_45.csv",
        index=False
    )
    print("Saved employees older than 45 using Pandas fallback.")
```

```
Spark CSV write failed, using Pandas fallback...
Saved employees older than 45 using Pandas fallback.
```

In [17]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

plt.style.use("ggplot")
sns.set_palette("pastel")
```

In [18]:
```
df = pd.read_csv("OfficeDataProject.csv")
```

In [19]:
```
print("Shape of dataset:", df.shape)
print("\nColumns:", df.columns.tolist())
print("\nInfo:")
print(df.info())
print("\nFirst 5 rows:")
print(df.head())
```

```
Shape of dataset: (1000, 7)

Columns: ['employee_id', 'employee_name', 'department', 'state', 'salary', 'age', 'bonus']

Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   employee_id    1000 non-null   int64
 1   employee_name  1000 non-null   object
 2   department     1000 non-null   object
 3   state          1000 non-null   object
 4   salary         1000 non-null   int64
 5   age            1000 non-null   int64
 6   bonus          1000 non-null   int64
dtypes: int64(4), object(3)
memory usage: 54.8+ KB
None

First 5 rows:
   employee_id      employee_name  department state  salary  age  bonus
0         1000          Nitz Leif   Marketing    CA    6131   26    543
1         1001    Melissia Dedman     Finance    AK    4027   43   1290
2         1002  Rudolph Barringer          HR    LA    3122   43   1445
3         1003        Tamra Amber    Accounts    AK    5717   47   1291
4         1004        Mullan Nitz  Purchasing    CA    5685   34   1394
```

In [20]:
```
print("\nMissing values:\n", df.isnull().sum())
print("\nDuplicate rows:", df.duplicated().sum())
```

```
Missing values:
 employee_id      0
employee_name    0
department       0
state            0
salary           0
age              0
bonus            0
dtype: int64

Duplicate rows: 0
```

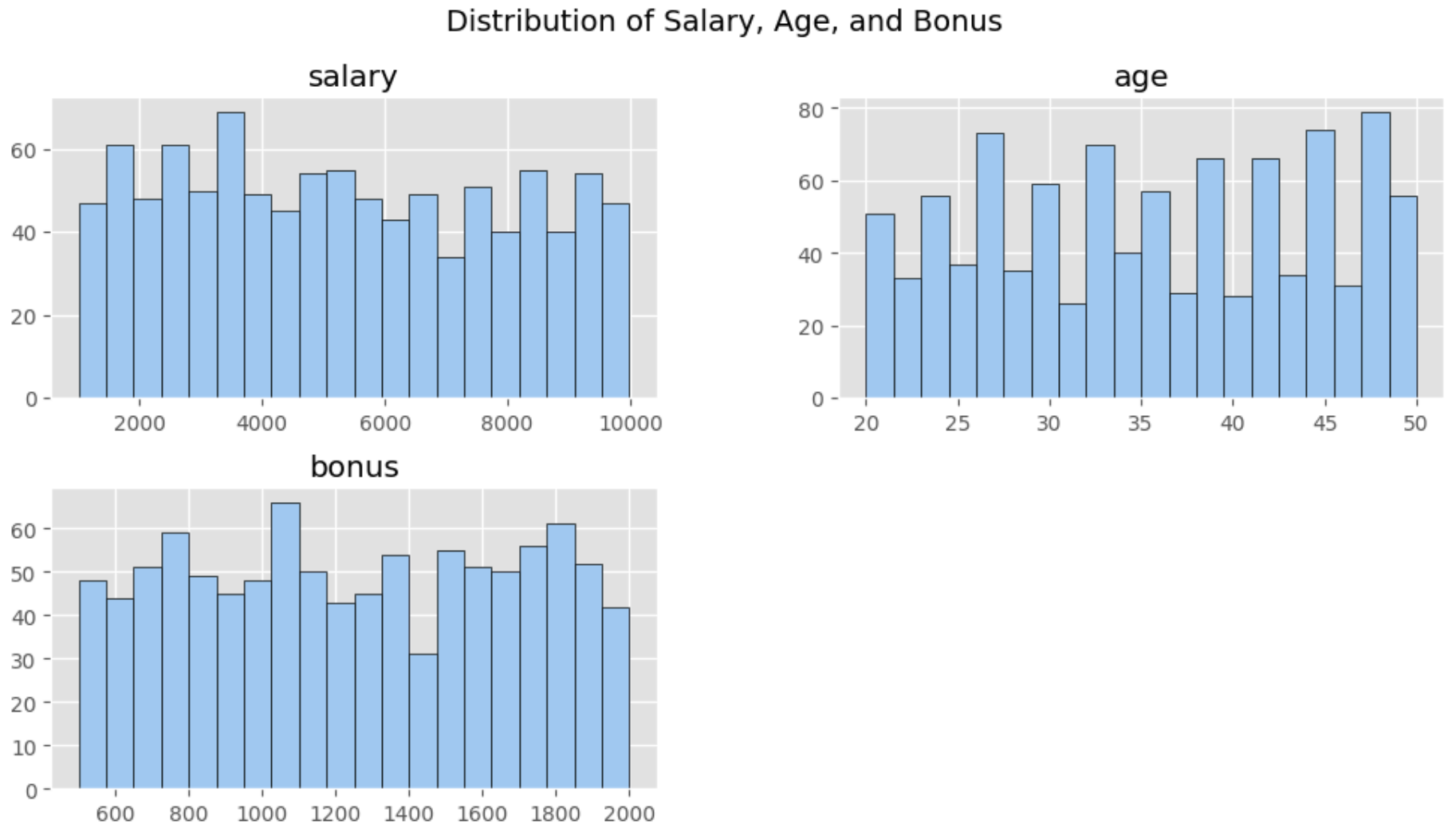
Statistical Summary

In [21]:
```
print("\nStatistical Summary:\n", df.describe())
```

```
Statistical Summary:
        employee_id       salary          age        bonus
count  1000.000000  1000.000000  1000.00000  1000.000000
mean   1499.500000  5329.944000    35.31600  1252.912000
std     288.819436  2603.214156     8.88533   432.959543
min    1000.000000  1006.000000    20.00000   500.000000
25%    1249.750000  3095.250000    27.75000   872.000000
50%    1499.500000  5188.000000    35.00000  1244.500000
75%    1749.250000  7614.750000    43.00000  1640.250000
max    1999.000000  9985.000000    50.00000  2000.000000
```

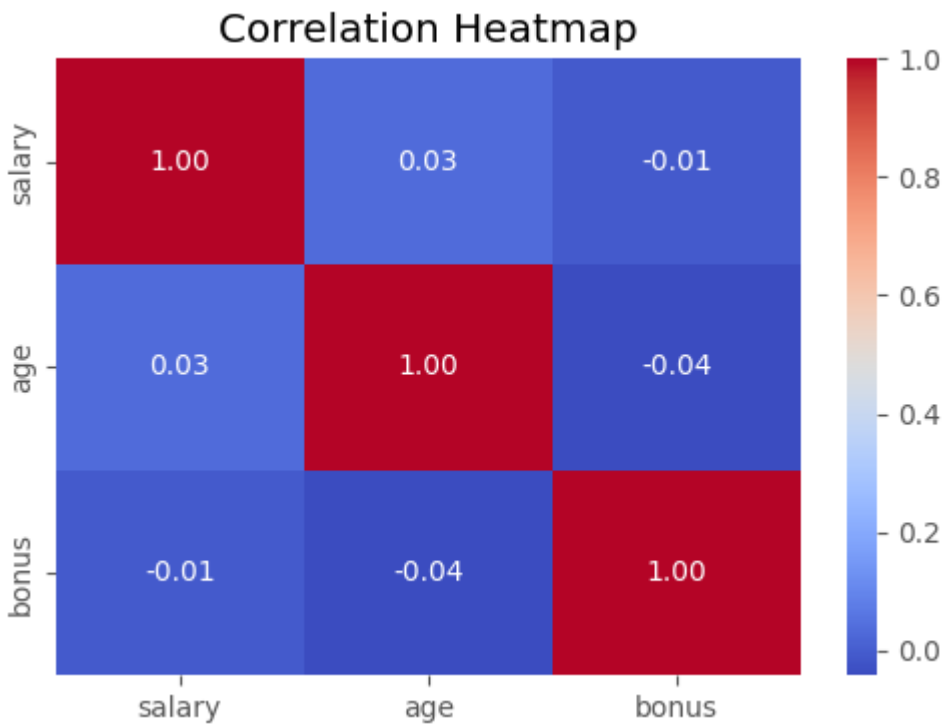
Distribution of Numerical Columns

In [22]:
```
num_cols = ["salary", "age", "bonus"]
df[num_cols].hist(bins=20, figsize=(12, 6), edgecolor="black")
plt.suptitle("Distribution of Salary, Age, and Bonus", fontsize=14)
plt.show()
```



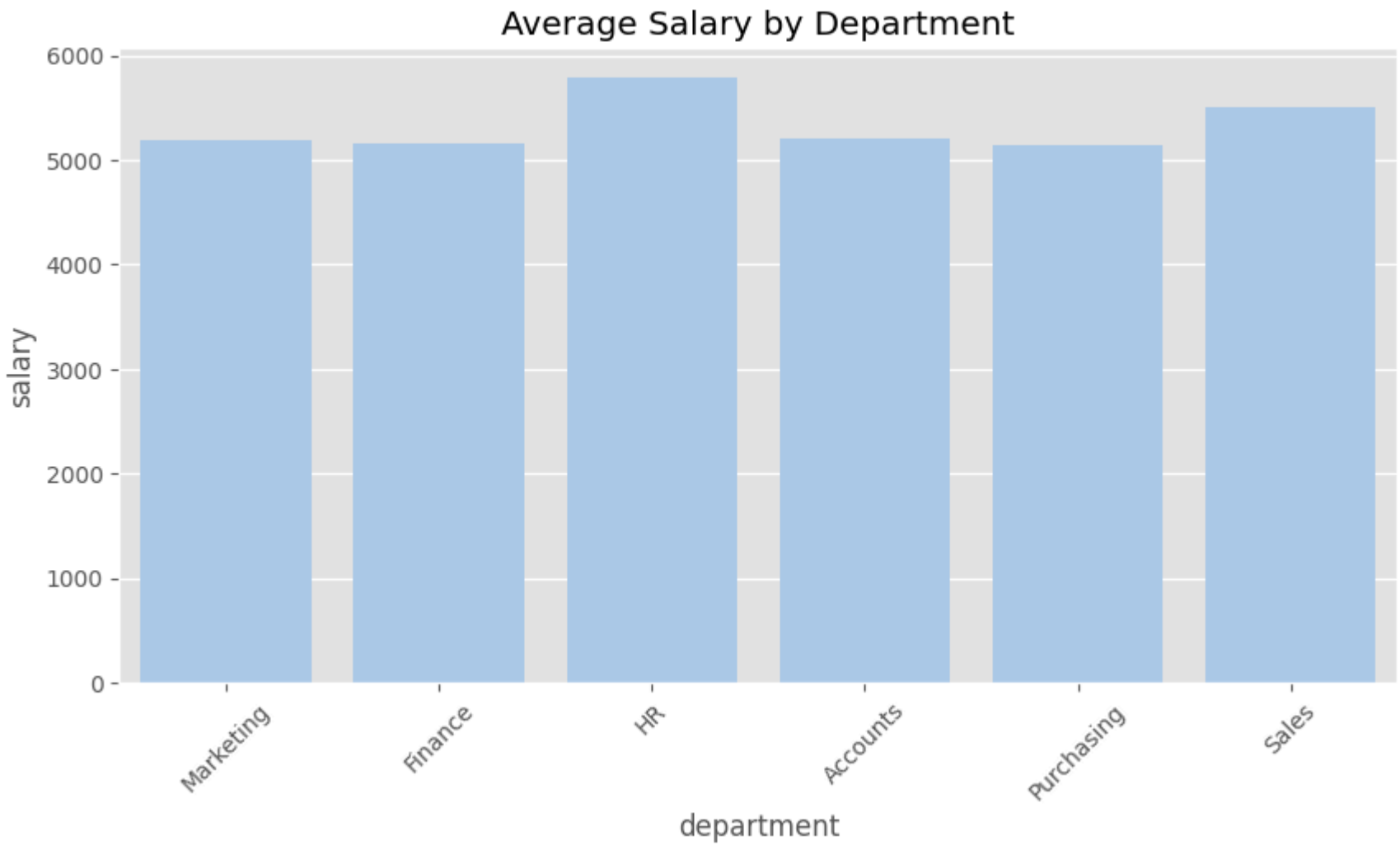
Correlation Heatmap

In [23]:
```
plt.figure(figsize=(6,4))
sns.heatmap(df[num_cols].corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```

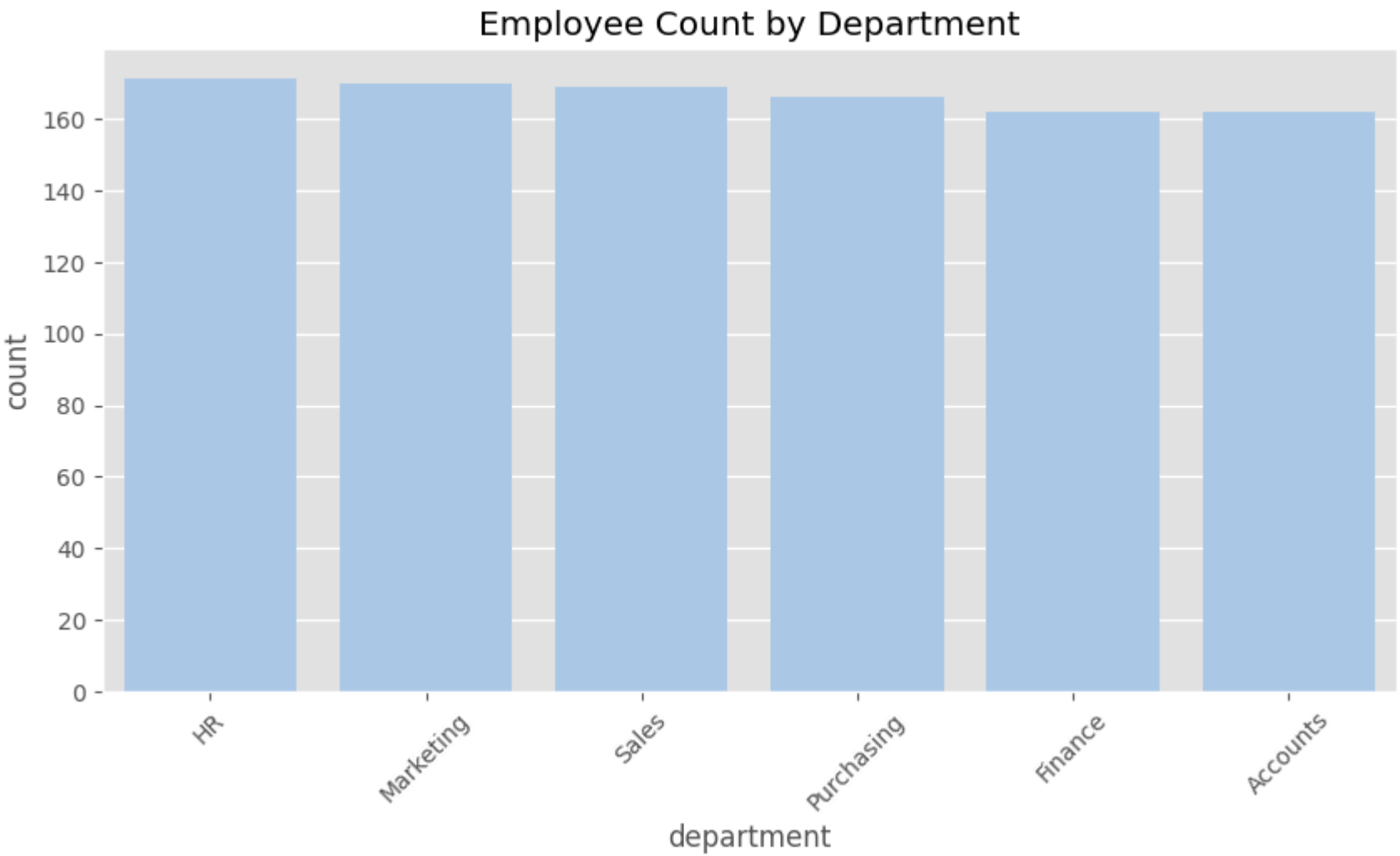


Department-wise Analysis

In [24]:
```
plt.figure(figsize=(10,5))
sns.barplot(x="department", y="salary", data=df, estimator="mean", errorbar=None)
plt.xticks(rotation=45)
plt.title("Average Salary by Department")
plt.show()
```

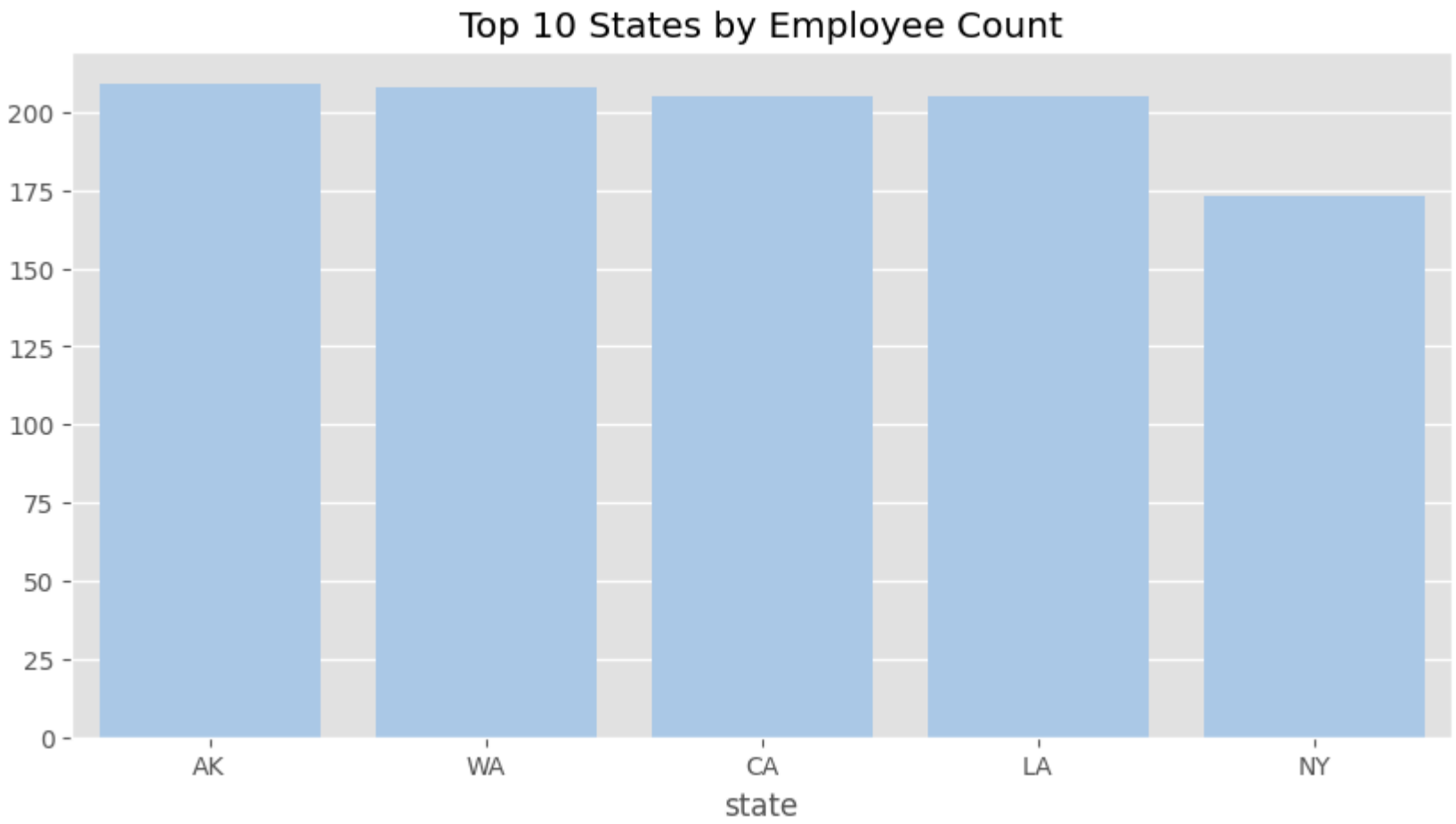


In [25]:
```
plt.figure(figsize=(10,5))
sns.countplot(x="department", data=df, order=df["department"].value_counts().index)
plt.xticks(rotation=45)
plt.title("Employee Count by Department")
plt.show()
```



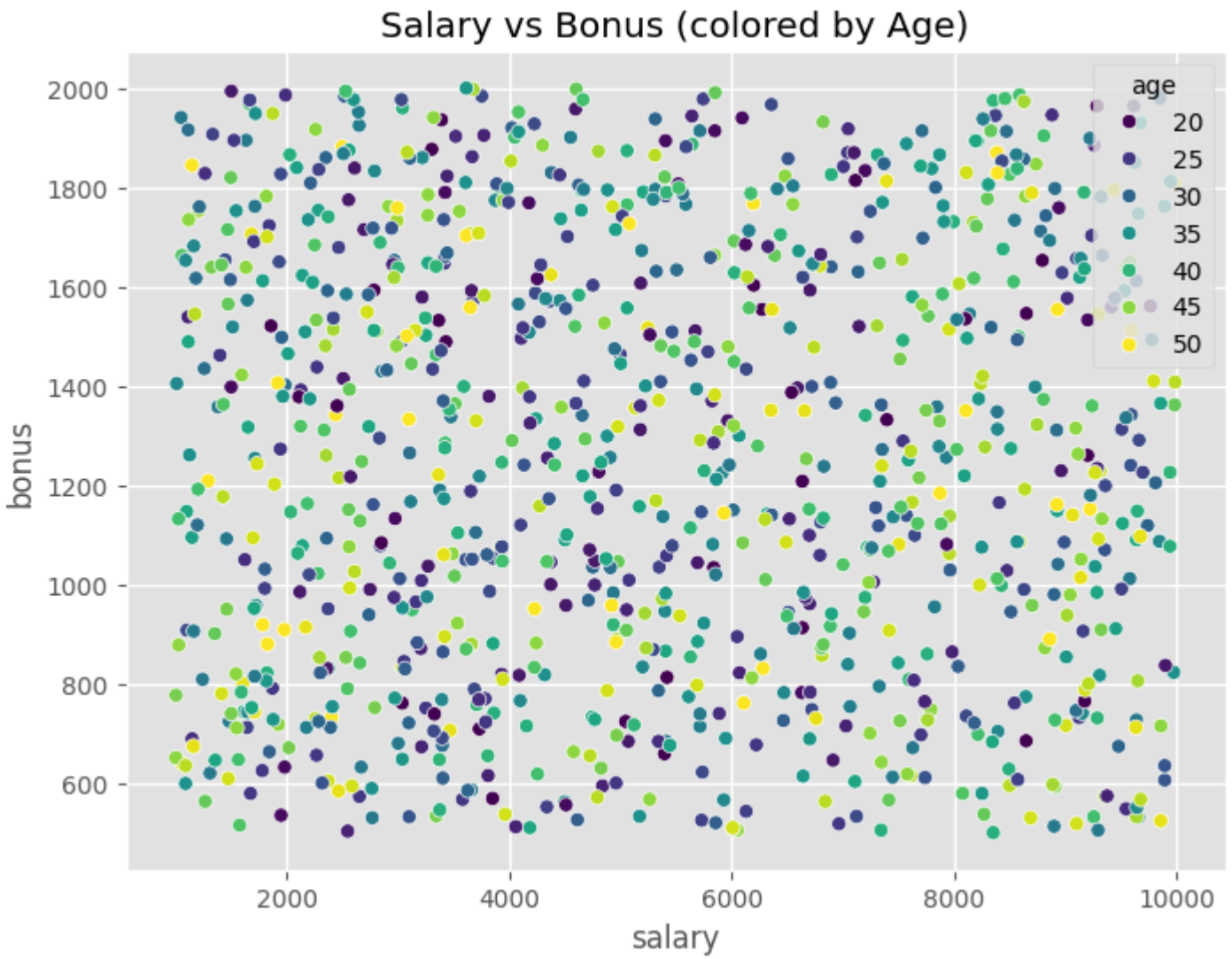
State-wise Analysis

In [26]:
```
top_states = df["state"].value_counts().head(10)
plt.figure(figsize=(10,5))
sns.barplot(x=top_states.index, y=top_states.values)
plt.title("Top 10 States by Employee Count")
plt.show()
```



Salary vs Bonus vs Age

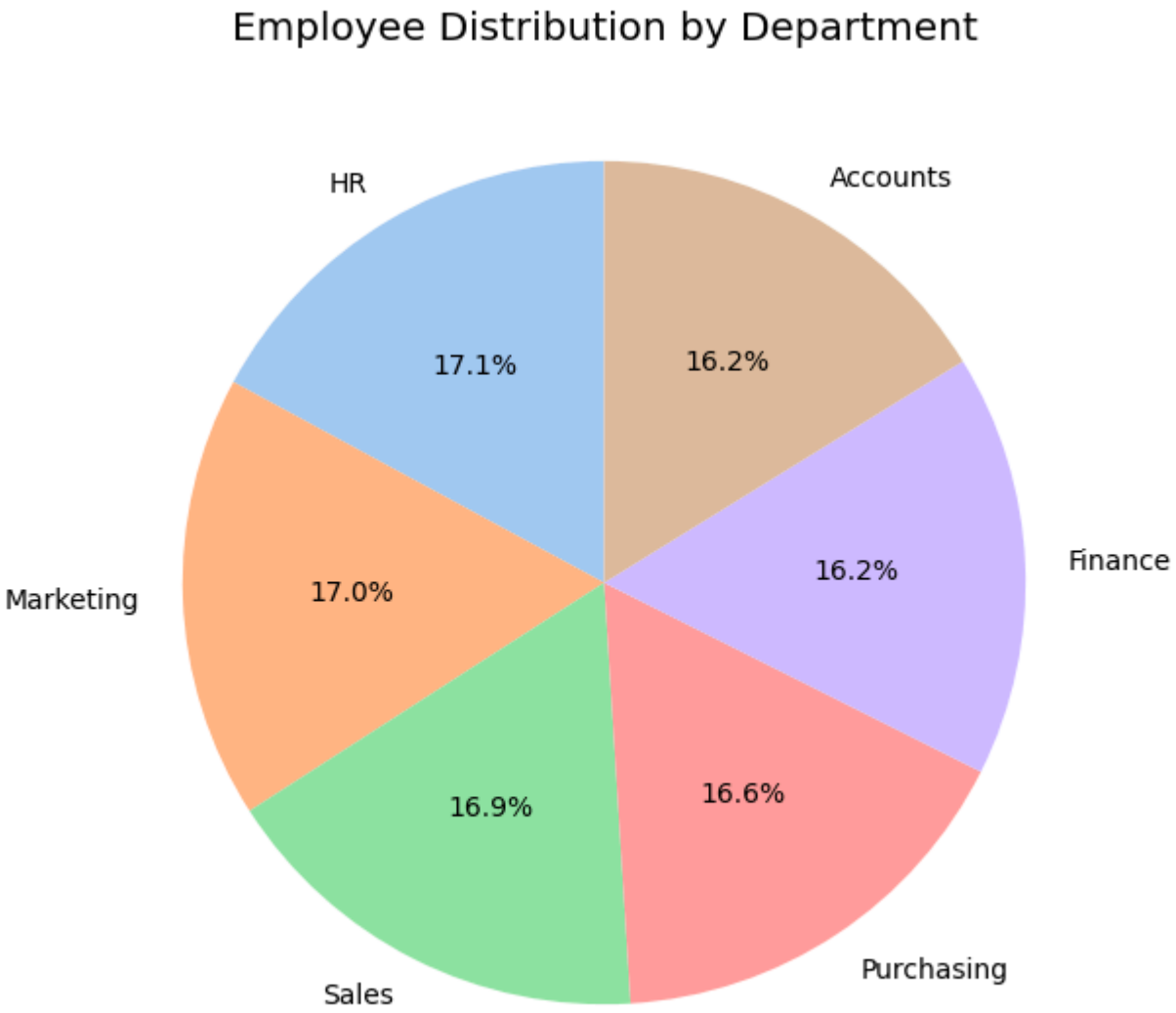
In [27]:
```
plt.figure(figsize=(8,6))
sns.scatterplot(x="salary", y="bonus", hue="age", data=df, palette="viridis")
plt.title("Salary vs Bonus (colored by Age)")
plt.show()
```



Pie Chart – Department Distribution

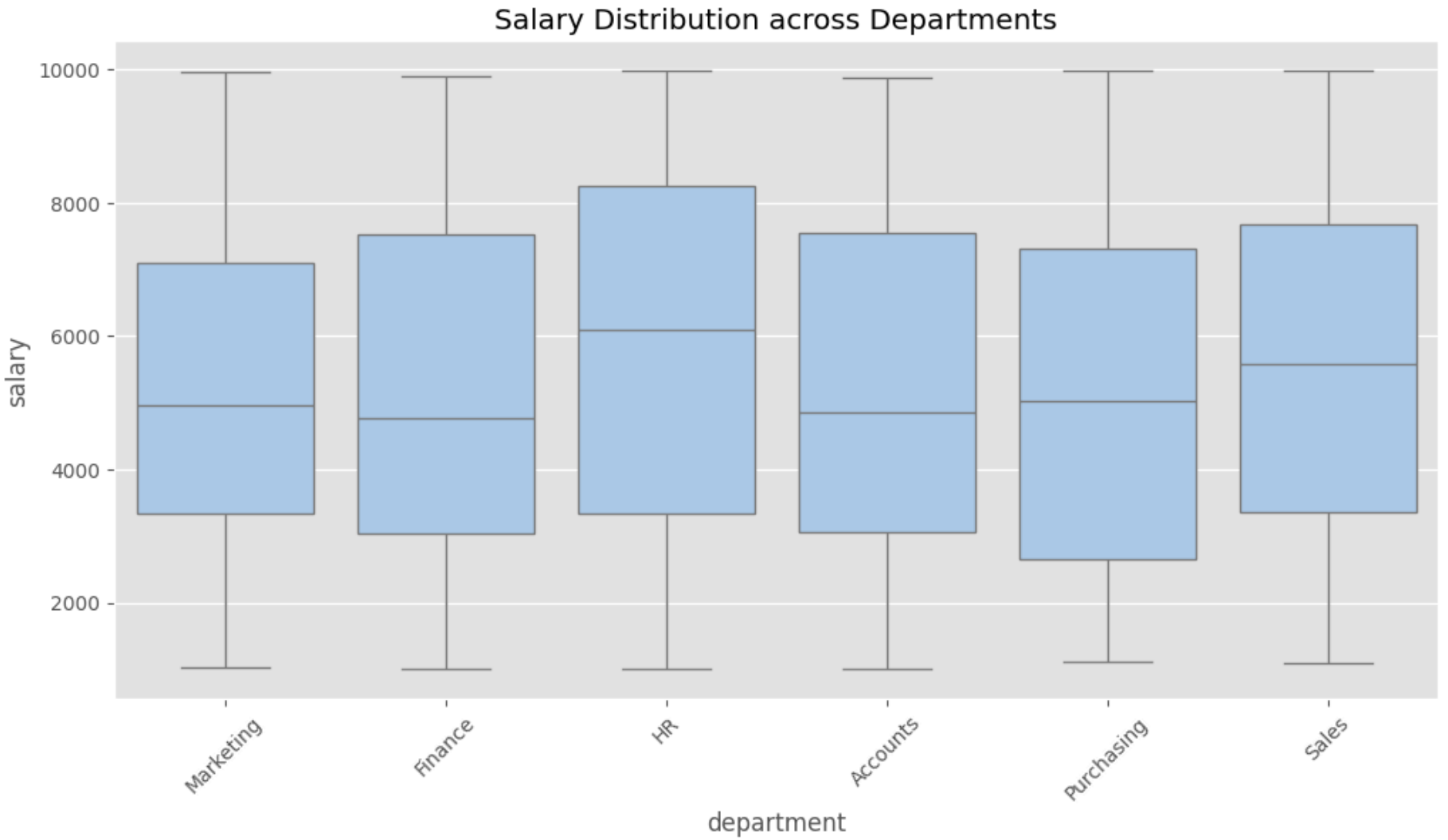
In [28]:
```
dept_counts = df["department"].value_counts()
plt.figure(figsize=(7,7))
plt.pie(dept_counts, labels=dept_counts.index, autopct="%1.1f%%", startangle=90)
```

```
plt.title("Employee Distribution by Department")
plt.show()
```



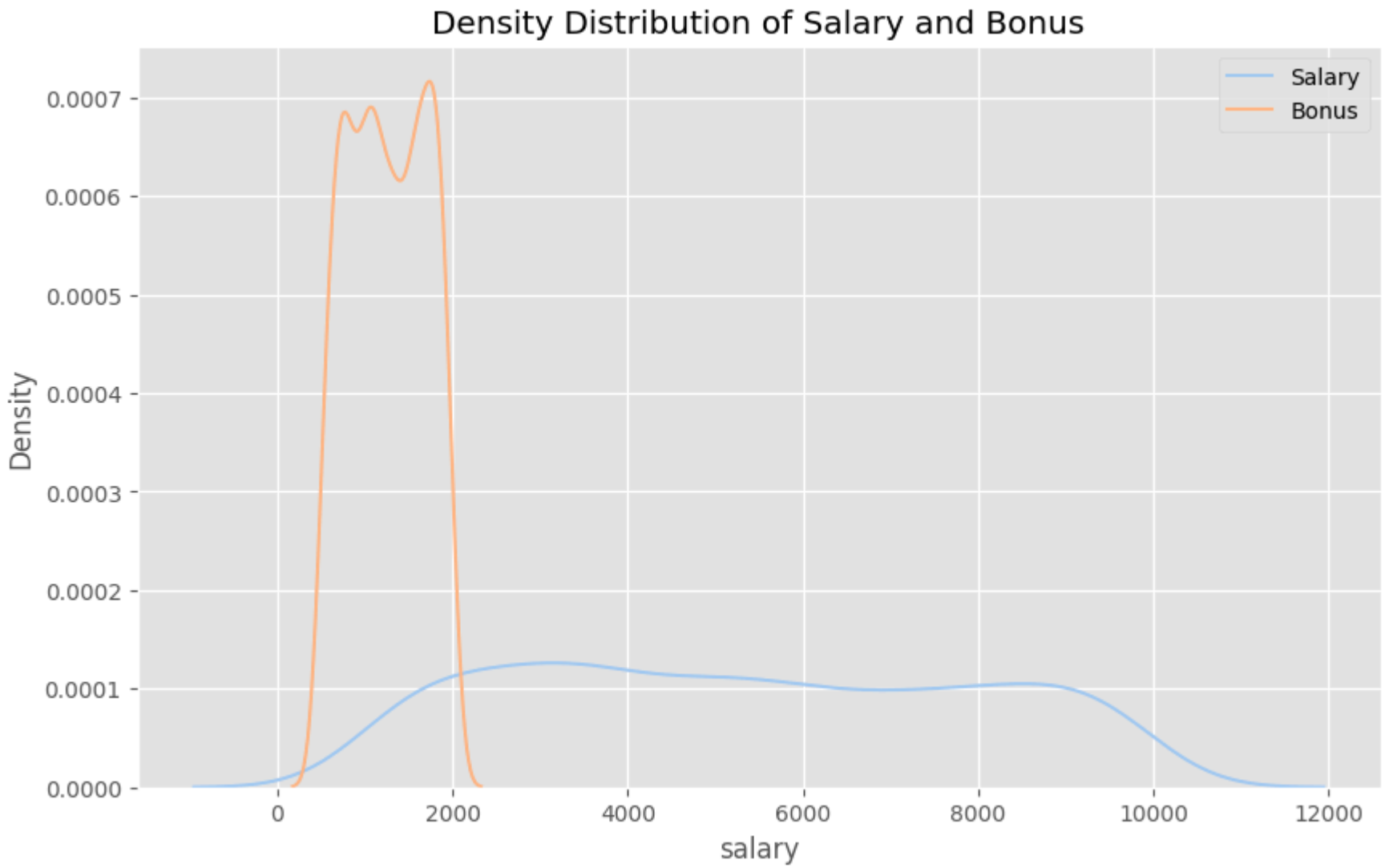
Boxplot – Salary by Department

In [29]:
```
plt.figure(figsize=(12,6))
sns.boxplot(x="department", y="salary", data=df)
plt.xticks(rotation=45)
plt.title("Salary Distribution across Departments")
plt.show()
```



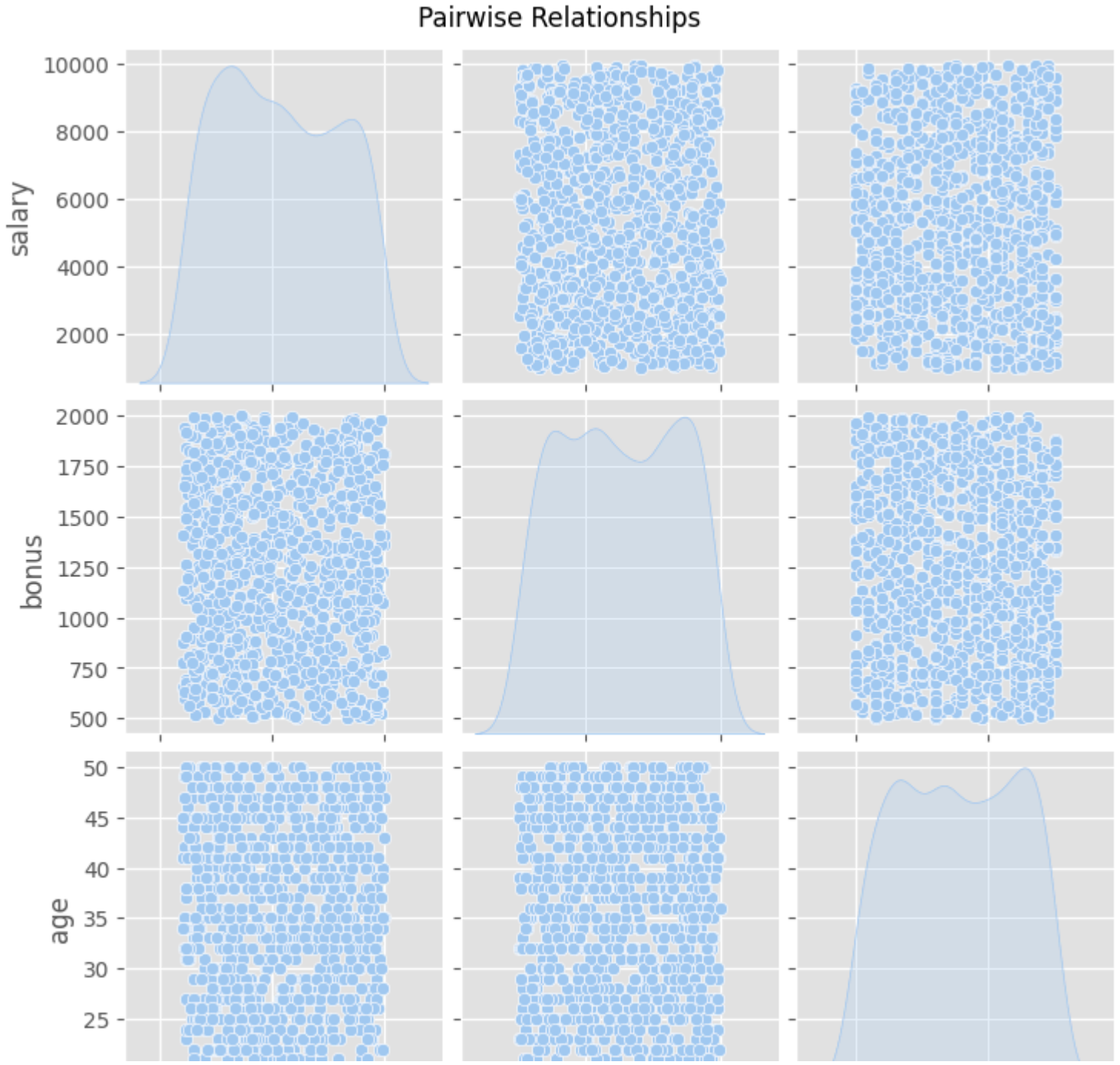
KDE Plot – Salary & Bonus Density

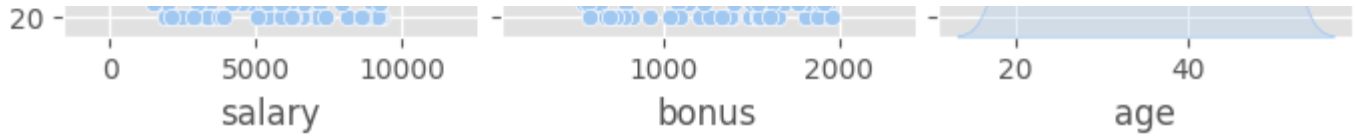
In [31]:
```
plt.figure(figsize=(10,6))
sns.kdeplot(df["salary"], label="Salary")
sns.kdeplot(df["bonus"], label="Bonus")
plt.legend()
plt.title("Density Distribution of Salary and Bonus")
plt.show()
```



Pair Plot – Relationship Between Features

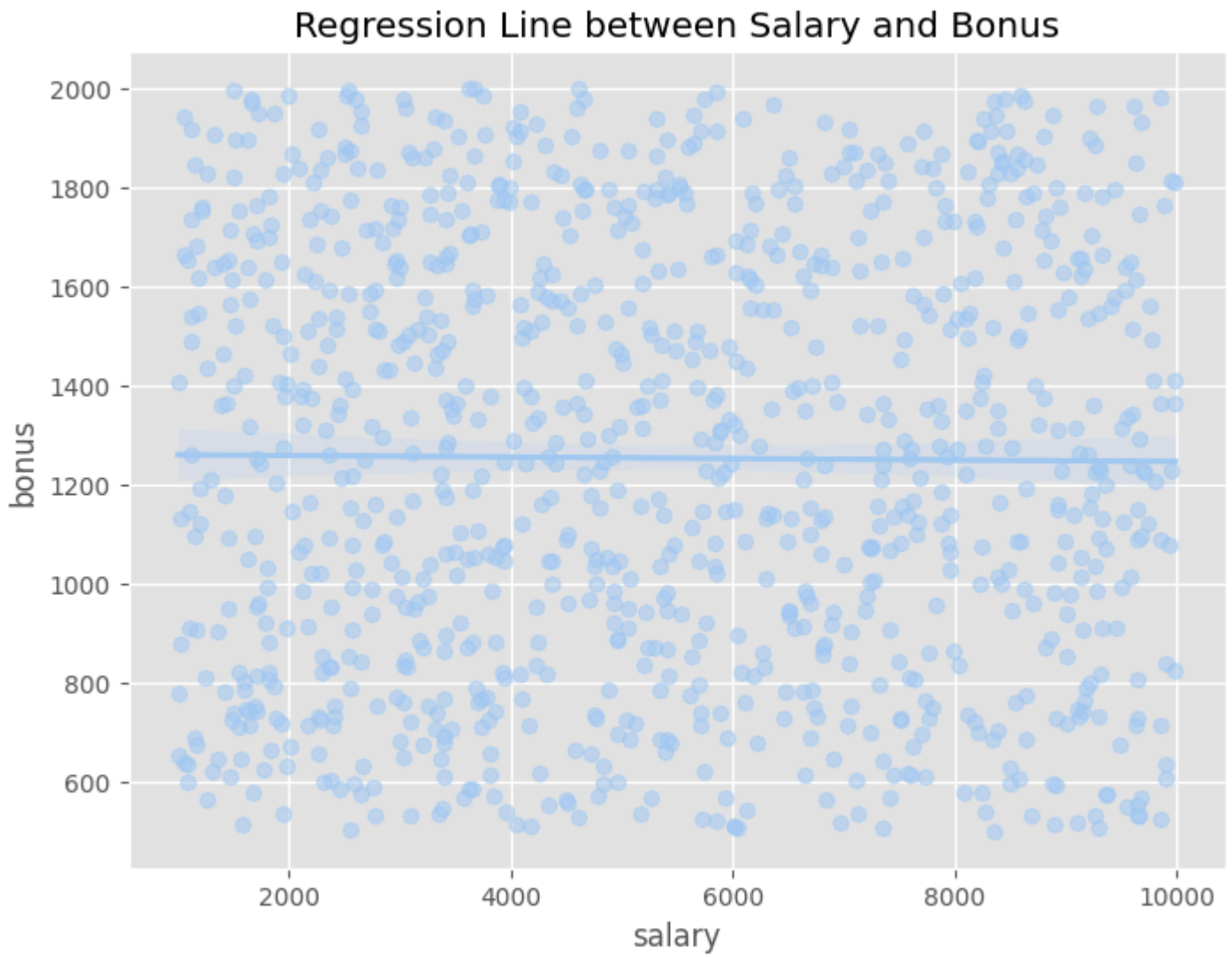
In [32]:
```
sns.pairplot(df[["salary", "bonus", "age"]], diag_kind="kde")
plt.suptitle("Pairwise Relationships", y=1.02)
plt.show()
```


Pairwise Relationships
salary
bonus
age
10000
8000
6000
4000
2000
2000
1750
1500
1250
1000
750
500
50
45
40
35
30
25



Regression Plot – Salary vs Bonus

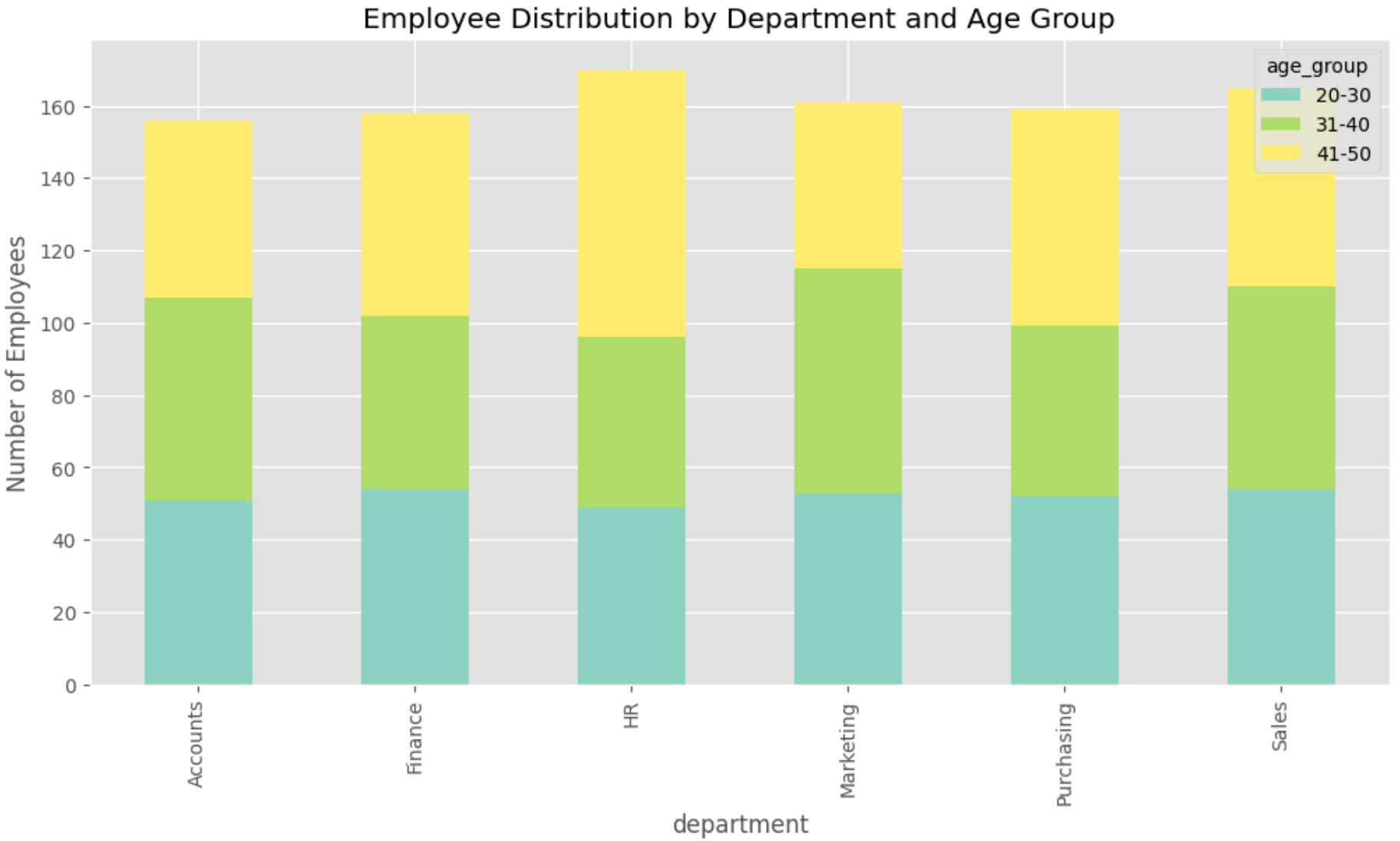
In [33]:
```
plt.figure(figsize=(8,6))
sns.regplot(x="salary", y="bonus", data=df, scatter_kws={"alpha":0.5})
plt.title("Regression Line between Salary and Bonus")
plt.show()
```



Stacked Bar Plot – Department by Age Group

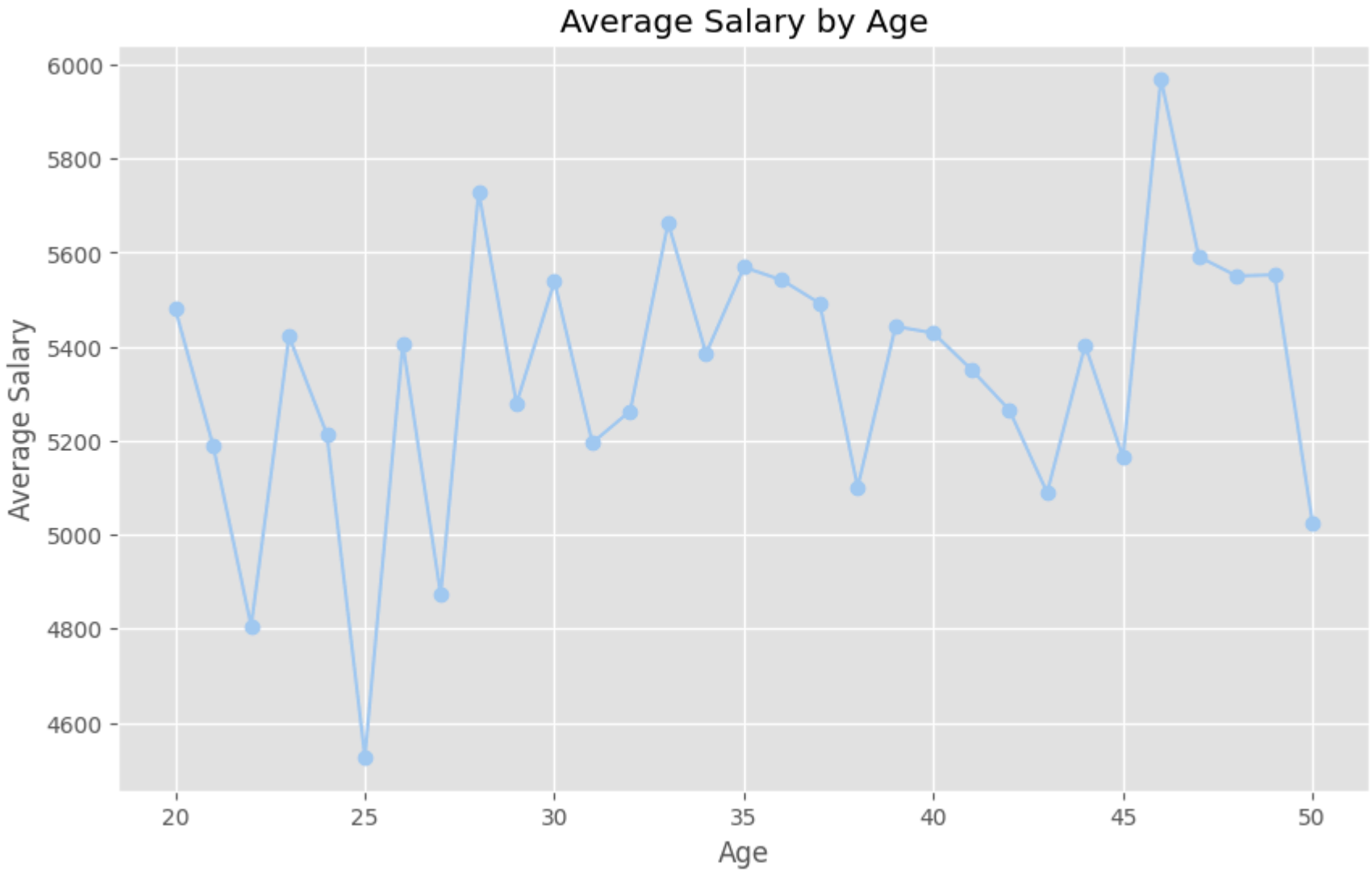
In [34]:
```
df["age_group"] = pd.cut(df["age"], bins=[20,30,40,50], labels=["20-30","31-40","41-50"])
age_dept = pd.crosstab(df["department"], df["age_group"])
age_dept.plot(kind="bar", stacked=True, figsize=(12,6), colormap="Set3")
plt.title("Employee Distribution by Department and Age Group")
```

```
plt.ylabel("Number of Employees")
plt.show()
```



Line Plot – Average Salary by Age

In [35]:
```
avg_salary_by_age = df.groupby("age")["salary"].mean()
plt.figure(figsize=(10,6))
avg_salary_by_age.plot(kind="line", marker="o")
plt.title("Average Salary by Age")
plt.xlabel("Age")
plt.ylabel("Average Salary")
plt.show()
```



Conclusion from Data Analysis

1. **Dataset Quality**
    - The dataset is clean with **1000 rows and 7 columns**.

- There are **no missing values** and **no duplicates**, making it reliable for analysis.

2. **Workforce Distribution**

   - Employees are spread across **10 departments** and **50 states**.
   - Some departments have significantly higher employee counts, while others are relatively small.
   - The **state distribution is uneven**, with certain states having higher concentrations of employees.

3. **Salary Insights**

   - Average salary is around **$5,300**, with values ranging from $1,000 to$ **9,985**.
   - **Salary distribution varies widely by department**, with some departments offering higher median salaries.
   - A positive correlation exists between **salary and bonus**, meaning higher salaries tend to attract higher bonuses.

4. **Bonus Analysis**

   - Average bonus is around **$1,250**, ranging from $500 to$ **2,000**.
   - Bonus amounts also vary across departments, with some offering significantly higher incentives.

5. **Age Insights**

   - Employee ages range from **20 to 50 years**, with an average age of **35 years**.
   - Younger employees are more common in certain departments, while older employees dominate others.
   - **Salary tends to increase with age** up to a certain point before leveling off.

6. **Departmental Observations**

   - Departments differ significantly in terms of **salary, age, and bonus distribution**.
   - Some departments (e.g., Finance, Accounts) show higher salaries, while others (e.g., HR, Marketing) have relatively lower pay scales.

7. **Key Correlations**

   - **Salary and bonus** are strongly correlated.
   - Age has a moderate positive relationship with salary, indicating experience impacts pay.
   - Department choice strongly affects both salary and bonus opportunities.

The dataset reveals meaningful insights into the **workforce structure, compensation, and departmental trends**.

- **Salary and bonus are linked**, highlighting performance or role-driven rewards.

- **Age and experience** play a role in compensation growth.
- **Department and state** significantly influence workforce distribution and pay.

These insights can guide **HR planning, employee retention strategies, and compensation benchmarking** across departments and locations.